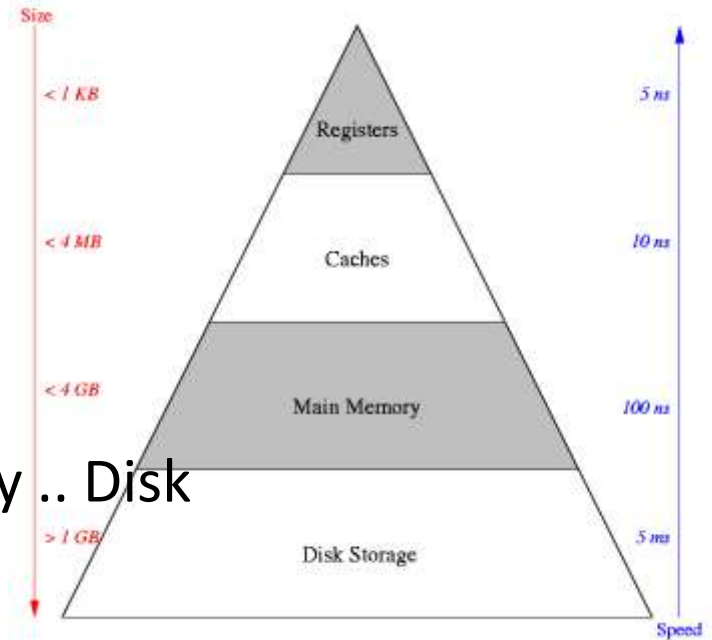


Lecture 10

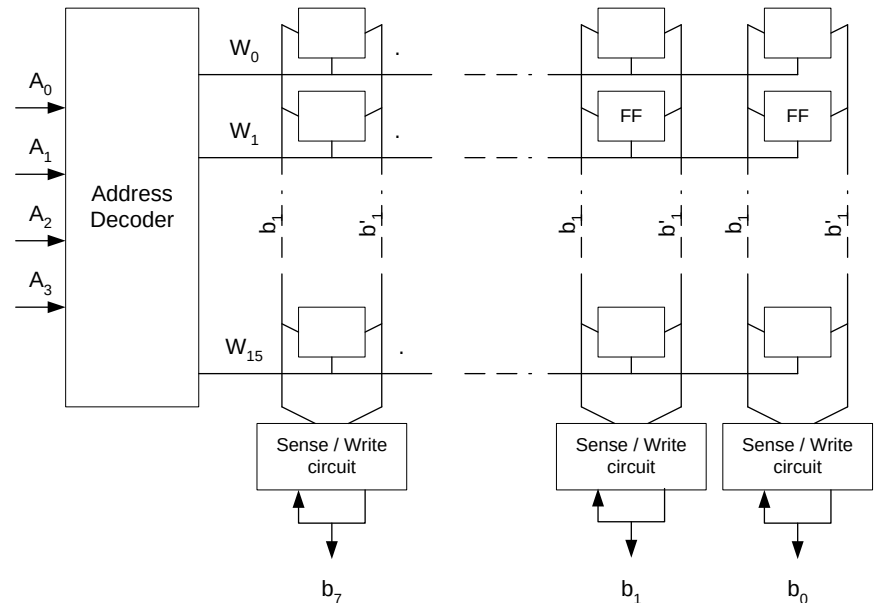
Cache Memory

Previously ..

- Memory Hierarchy
 - Register .. Cache .. Main Memory .. Disk
- Memory System
 - RAM, ROM, CACHE, and Virtual Memory

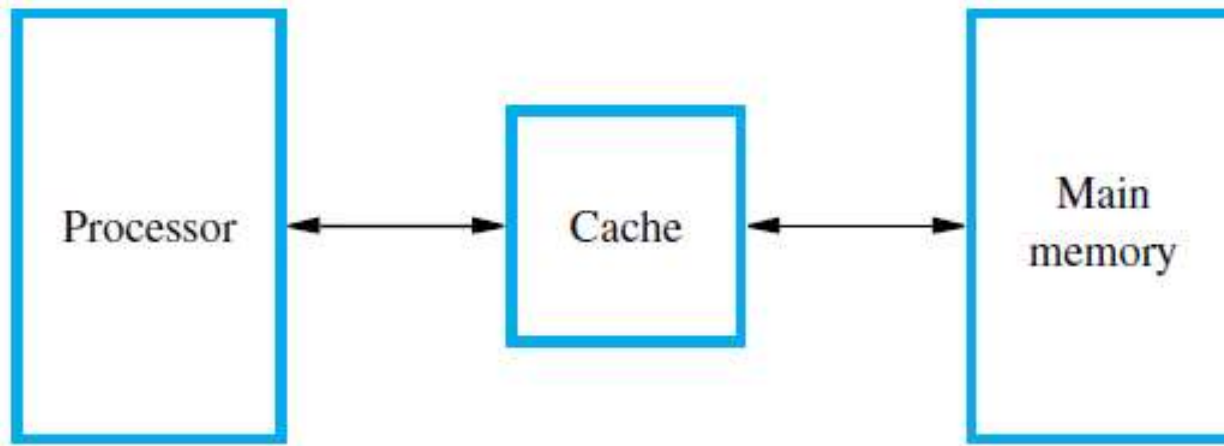


- Memory Architecture
 - RAM and ROM



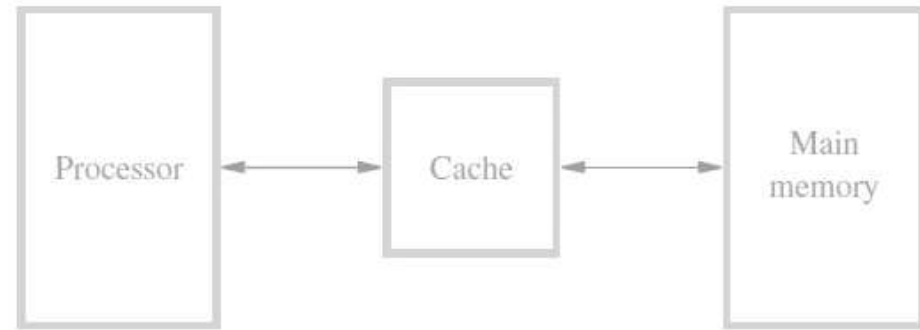
Basic concepts

- Its purpose is to make the main memory appear to the processor to be much faster than it actually is.



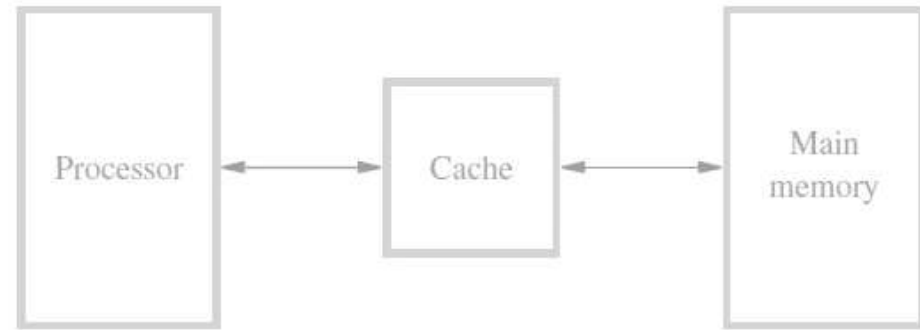
The use of a cache memory

Locality of Reference



- Caches are used to store parts of program that are **frequently accessed**.
- A small part of a program is usually executed several times, while the remainder of the code may be accessed relatively infrequently. This is referred to as *locality of reference*, which manifests itself in two ways.
 - *Temporal* : a **recently executed** instruction is likely to be executed again soon.
 - *Spatial* : instructions in **close proximity** to a recently executed instruction are also likely to be executed soon.

Cache Hit

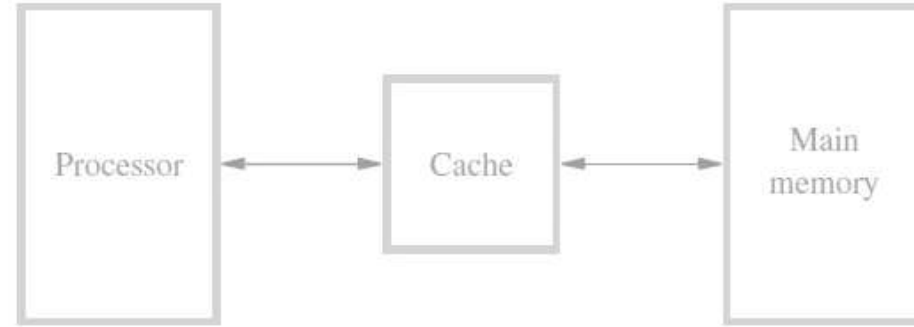


- The read or write operation is performed on the appropriate cache location.
- Read operation: The processor reads from a cache, and the main memory is not involved.
- Write operation: The main memory is involved.
 - **Write through protocol:** the cache location and the memory location are updated simultaneously.
 - **Write back protocol:** The cache location is updated and is marked as 'update' with an associated flag bit called, *the dirty or modified bit*. The main memory data will be updated later, when this word is to be removed from a cache.

Write Through vs Write Back Protocol

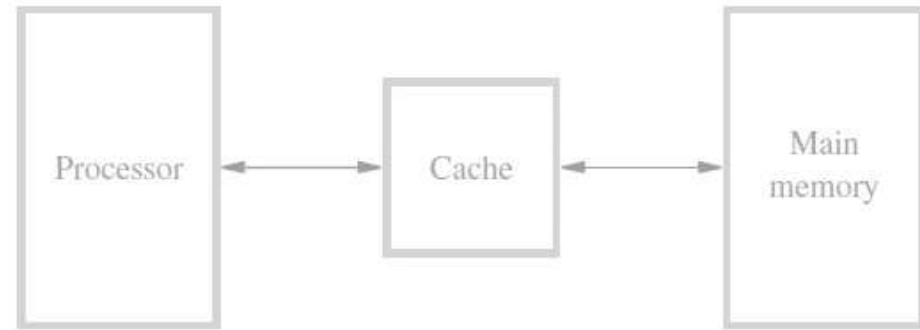
- The write-through protocol is simpler than the write-back protocol, but it results in **unnecessary Write operations** in the main memory when a given cache word is updated several times during its cache residency.
- The write-back protocol also involves unnecessary Write operations, because all words of the block are eventually written back, even if only a single word has been changed while **the block** was in the cache.
- The write-back protocol is used most often, to take advantage of the high speed with which data blocks can be transferred to memory chips.

Cache Miss



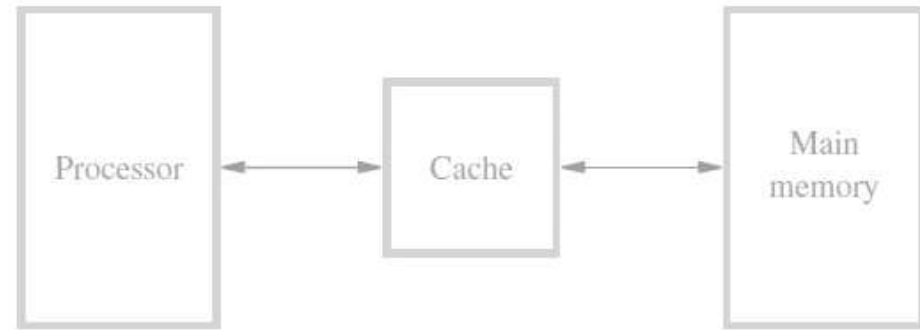
- Read miss occurs when the addressed word in an operation is not in the cache. After the entire block is loaded into the cache, the requested particular word will be forwarded to the CPU.
- Alternatively, this word may be sent to the processor as soon as it is read from the main memory. The latter approach, which is called **load-through protocol** reduces the processor's waiting time somewhat, at the expense of more complex circuitry.
- When a Write miss occurs in a computer that uses the **write-through protocol**, the information is written directly into the main memory.

Other Issues on Cache Miss



- In a **pipelined processor** can cause instruction execution to **stall** for one or more cycles.
- This can occur if a Load or Store instruction requests access to data in the memory at **the same time** that a subsequent instruction is being fetched. When this happens, instruction fetch is delayed until the data access operation is completed.
- To avoid stalling the pipeline, many processors use **separate caches for instructions and data**, making it possible for the two operations to proceed in parallel.

Cache Memory

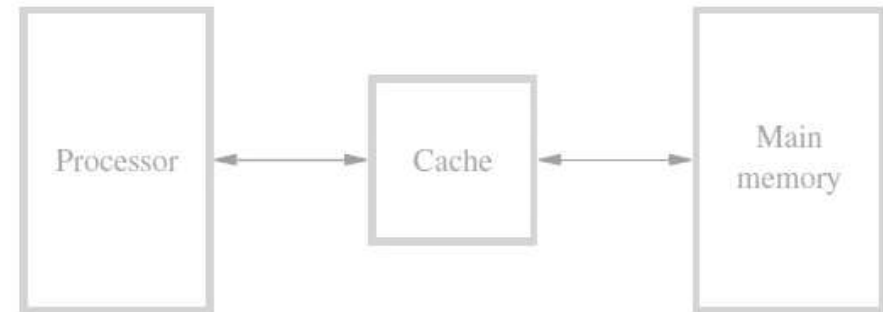


- “Block” refers to a set of contiguous address.
- The number of blocks in cache is small compared to the total number of blocks in the main memory. The correspondence between the main memory blocks and those in the cache is specified by the **mapping function**.
- The cache control hardware must decide which block should be removed to create space for the new block. This decision constitutes the **replacement algorithm**.

Memory Mapping

- Memory mapping refers to methods for specifying where memory blocks are placed in the cache. There are three methods as described below,

- Direct mapping
- Associative mapping
- Set-associative mapping



- Example

- A cache consists of 128 blocks of 16 words each.
- The main memory is addressable by a 16 bit address. It has 64K words which can be viewed as 4K blocks of 16 words each.

Direct Mapping (1)

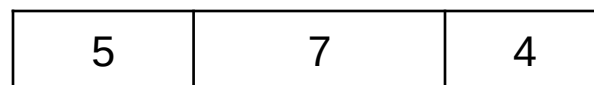
■ Memory Address

5	7	4
Tag	Block	Word

- 'Tag' is used to identify which block from a memory is currently in a cache.
- 'Block' is used to select a block in a cache.
- 'Word' is used to select a word in a block.

Direct Mapping (2)

An address (7-bit in the middle) generated by a processor will point to a position in a cache. The cache controller will go to the specified position and check tag bits. If the tag bits match, then the specified word can be read or written. **If the tag bits do not match, a cache miss will occur.**

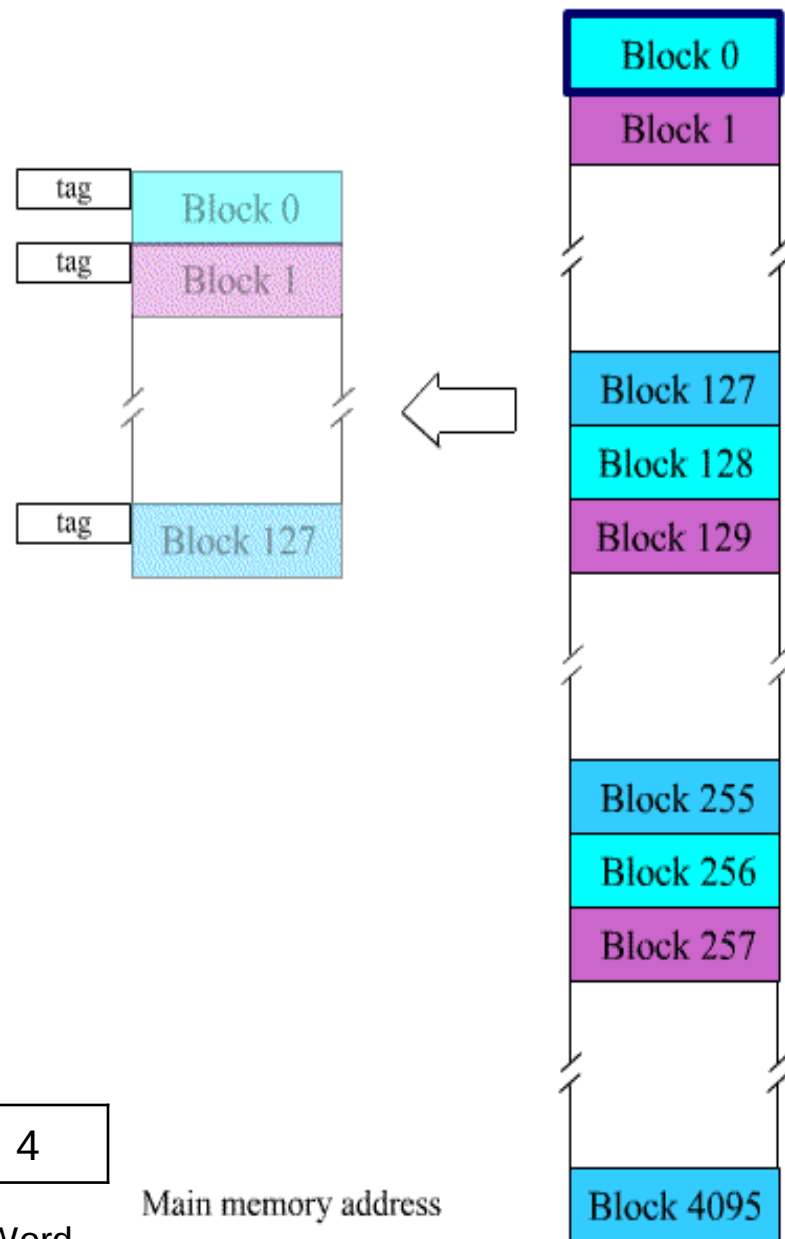


Tag

Block

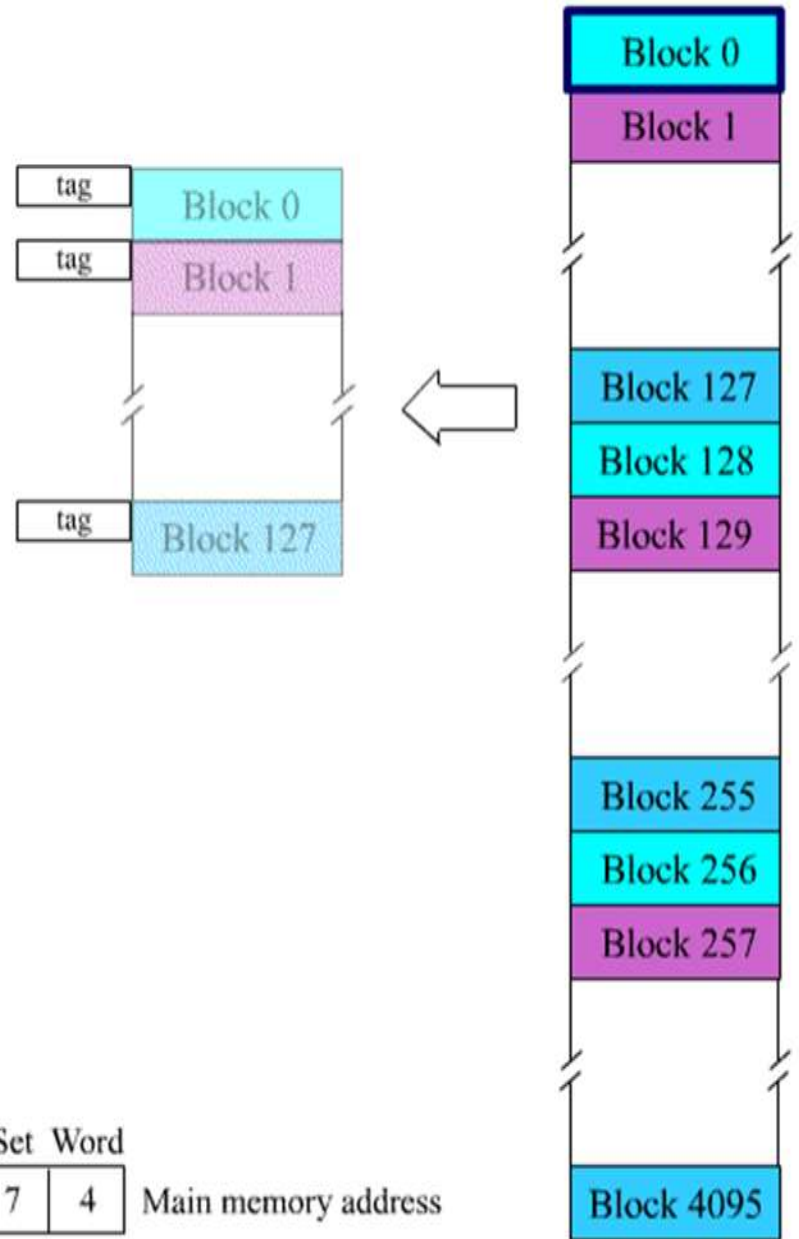
Word

Main memory address



Direct Mapping (3)

- Block j in a main memory will be mapped into 'block j modulo number of block' in a cache.
- Several blocks of the memory are mapped into the same block in a cache. Collision can occur even when the cache is not full.
- Direct mapping is efficient, but not flexible.



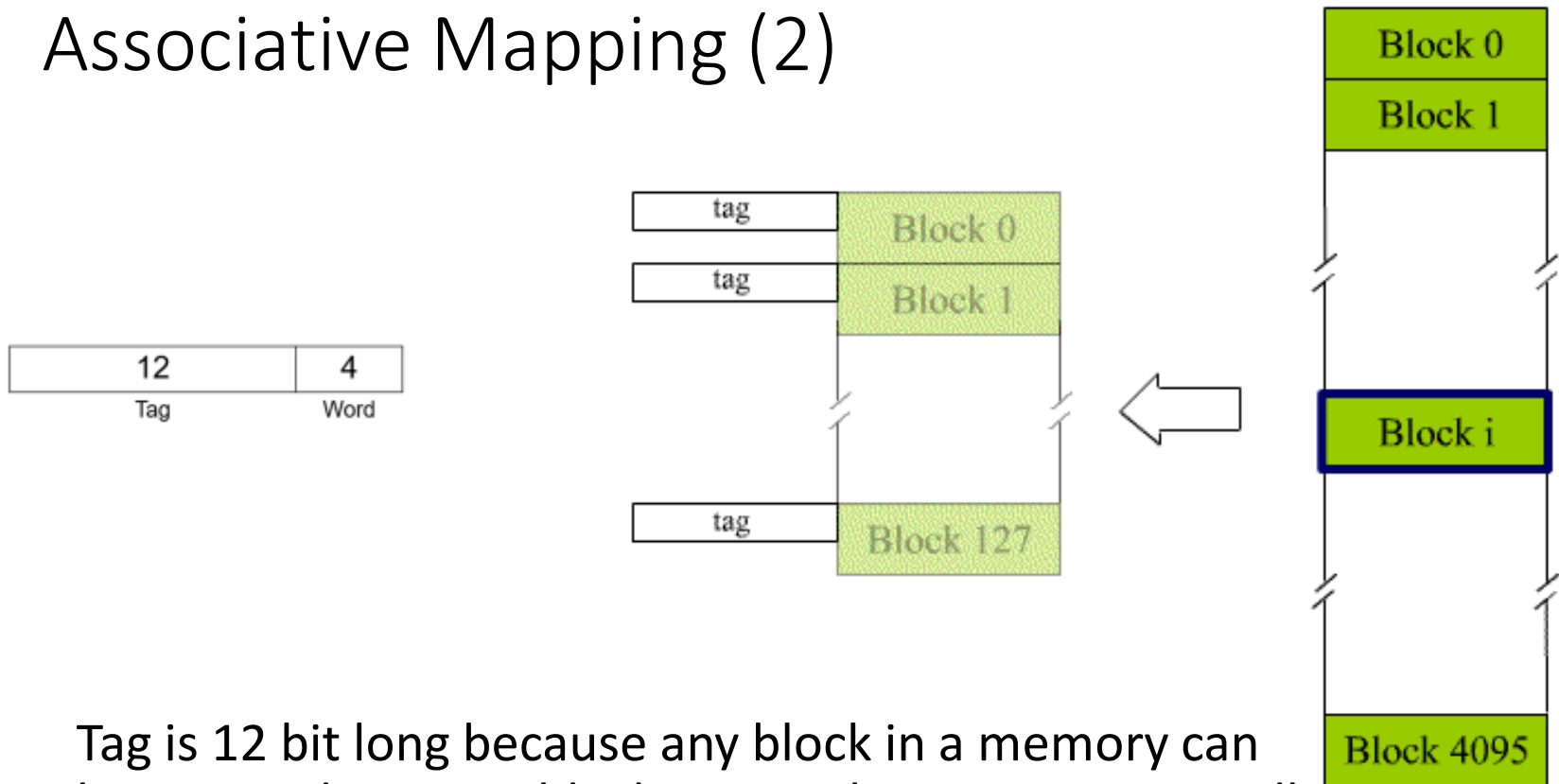
Associative Mapping (1)

■ Memory Address

12	4
Tag	Word

- 'Tag' is used to identify which block from s memory is currently in a cache.
- 'Word' is used to select a word in a block.

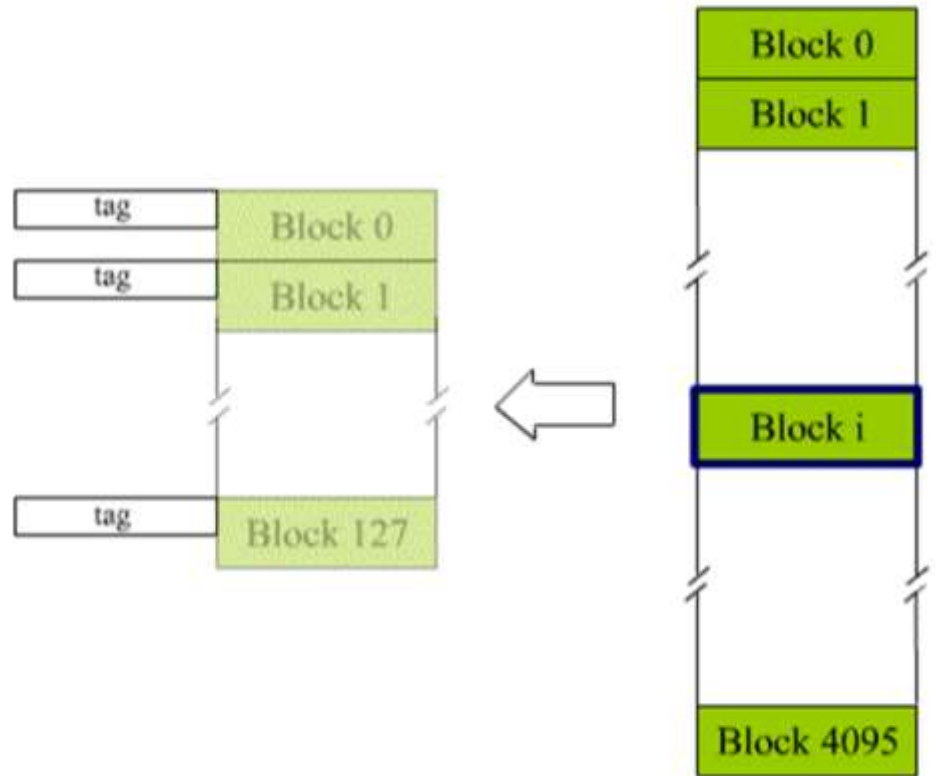
Associative Mapping (2)



Tag is 12 bit long because any block in a memory can be mapped into any block in a cache. A processor will compare tag from the processor-generated address with the tag in a cache one by one until the block is found or the whole cache is searched.

Associative Mapping (3)

- In Associative Mapping, a main memory block can be mapped into any cache block position.
- This mapping method is flexible but not efficient.



Set-Associative Mapping (1)

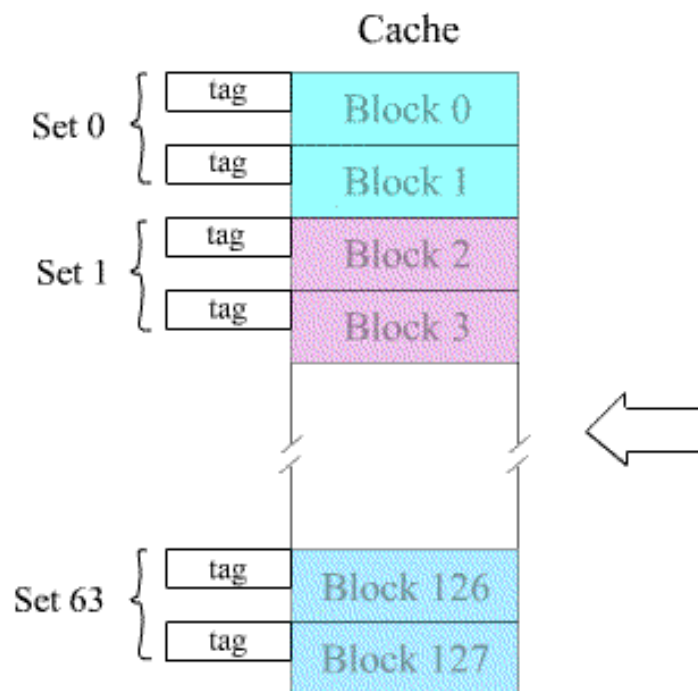
■ Memory Address

6	6	4
Tag	Set	Word

- 'Tag' is used to identify which block from a memory is currently in a cache.
- 'Set' is used to select a set in a cache.
- 'Word' is used to select a word in a block.

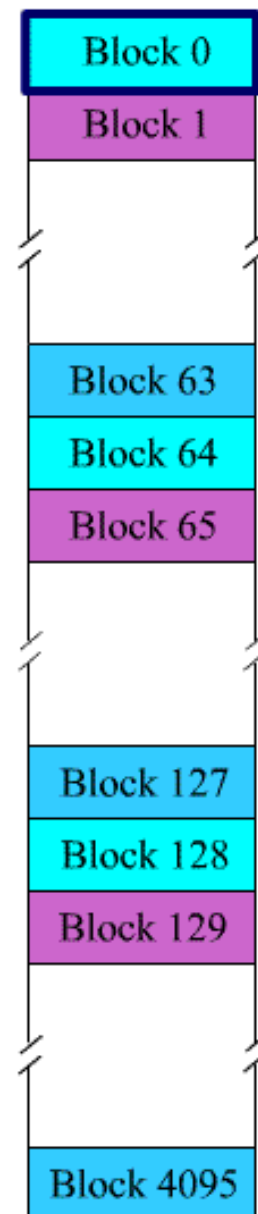
Set-Associative Mapping (2)

- Memory blocks 0, 64, 128, . . . , 4032 map into cache set 0, and they can occupy either of the two block positions within this set.
- Having 64 sets means that the 6-bit set field of the address determines which set of the cache might contain the desired block.



Tag	Set	Word	Main memory address
6	6	4	

Main memory



Set-Associative Mapping (3)

- In Set-Associative Mapping, cache blocks will be grouped into sets. Block j in a main memory will be mapped into 'block j modulo number of set' in a cache. Within a set, block j can be mapped into any cache block.
- The tag field of the address must then be associatively compared to the tags of the two blocks of the set to check if the desired block is present. This two-way associative search is simple to implement.
- The number of blocks per set is a parameter that can be selected to suit the requirements of a particular computer.

Set-Associative Mapping (4)

- For the main memory and cache sizes in above example, four blocks per set can be accommodated by a 5-bit set field, eight blocks per set by a 4-bit set field, and so on.
- The extreme condition of 128 blocks per set requires no set bits and corresponds to the fully-associative technique, with 12 tag bits. The other extreme of one block per set is the direct-mapping method.
- A cache that has k blocks per set is referred to as a k -way set-associative cache.

Valid Bit

- Valid bit is a control bit in a cache indicating whether a data block in a cache is valid.
 - Valid bit is set to 0 initially.
 - When data is loaded into a cache from a memory for the first time, the valid bit is set to 1.
 - When there is an update to data in the memory bypassing the cache, the valid bit in a cache will be cleared.
 - If the invalid block is to be accessed, the block will be reloaded from a memory first.

Replacement Algorithms

- In **associative and set-associative mapping** methods, when a cache is full, the cache controller must decide which one of the old blocks is to be overwritten. The followings are the example of the replacement algorithms.
 - Least recently used (LRU)
 - Remove the oldest block
 - Random Selection

Least Recently Used (LRU)

- When a block is to be overwritten, it is sensible to overwrite the one that has gone the longest time without being referenced. This block is called the least recently used block.

Performance Considerations

- Performance of a processor depends on:
 - How fast machine instructions can be brought into the processor for execution.
 - How fast the instructions can be executed.
- The main purpose of this hierarchy is to create a memory that the processor sees as having a short access time and a large capacity.
 - Caches improve performance is dependent on how frequently the requested instructions and data are found in the cache.

Hit Rate and Miss Penalty (1)

- Hit rate measures how efficient the memory system is. We measure the success rate in accessing the information in the memory hierarchy.
- Miss penalty is the time used to transfer data from a slower memory to a faster memory in a hierarchy.
- Note: **Memory interleaving** can reduce miss penalty.

Hit Rate and Miss Penalty (2)

t_{ave} is the average time to access information

$$t_{ave} = ht_C + (1-h)t_M$$

h is a hit rate

t_C is an access time of the cache

t_M is a miss penalty (an access time of the mem?)

Example Problem (1)

- Assume that
 - The time used to access the main memory is 10 cycles.
 - The cache is 4-way set-associative that can hold 8 blocks.
 - A cache miss costs 17 cycles.
 - The time used to access data in a cache is 1 cycle.
 - The **hit rate** of the cache is **0.95** for instruction loading and **0.9** for data loading.
 - 30% of instructions are read or write operations.

Example Problem (2)

- The time used to access the main memory is 10 cycles.
- The cache is 4-way set-associative that can hold 8 blocks.
- A cache miss costs 17 cycles.
- The time used to access data in a cache is 1 cycle.
- The hit rate of the cache is 0.95 for instruction loading and 0.9 for data loading.
- 30% of instructions are read or write operations.

- Without cache for 100 instructions

$$\begin{aligned} t_{avg} &= (100+30) * 10 \\ &= 1300 \text{ cycles} \end{aligned}$$

- With cache: An average time

$$\begin{aligned} t_{avg} &= 100*(0.95*1 + 0.05*17) \\ &\quad + 30*(0.9*1 + 0.1*17) \\ &= 260 \text{ cycles} \end{aligned}$$

- Implementing a 1 level cache (4-way interleaving), can save the accessing time by 5 times.

Example Problem (3)

- Ideal case (100% hit rate)

$$\begin{aligned} t_{ave} &= (100+30) * 1 \\ &= 130 \text{ cycles} \end{aligned}$$

- We can see that by using a 1 level cache, the performance of the system is only 2 times slower than the ideal case.

Multi-level Cache

- In case of a 2-level cache:

$$t_{ave} = h_1 t_{C1} + (1-h_1)h_2 t_{C2} + (1-h_1)(1-h_2)t_M$$

h_1 and h_2 are hit rates in the primary and the secondary cache.

t_{C1} and t_{C2} are access times in the primary and the secondary cache.

t_M is an access time in the main memory (miss penalty).

- In case of a 3-level cache:

$$t_{ave} = h_1 t_{c1} + (1-h_1)h_2 t_{c2} + (1-h_1)(1-h_2)h_3 t_{c3} + (1-h_1)(1-h_2)(1-h_3)t_m$$

Example Problem: Multi-level Cache (1)

- Assume that
 - A 2-level cache, where each cache block has 8 words
 - The hit rate is the same for both level: data cache = 0.9 and instruction = 0.95.
 - The time to access 1 block for L1 is 1 cycle and L2 is 10 cycles.
 - 30% of instructions are read or write operations.
 - A cache miss costs 17 cycles.

Example Problem : Multi-level Cache (2)

- The average time:

$$\begin{aligned}t_{avg} &= ht_{c1} + (1-h)ht_{c2} + (1-h)^2t_m \\&= (0.95*1 + 0.05*0.95*10 + 0.0025*17) \\&\quad + 30*(0.9*1 + 0.1*0.9*10 + 0.001*17) \\&= 2.0585 \text{ cycles}\end{aligned}$$

- Without interleaving modules, the average access time would be:

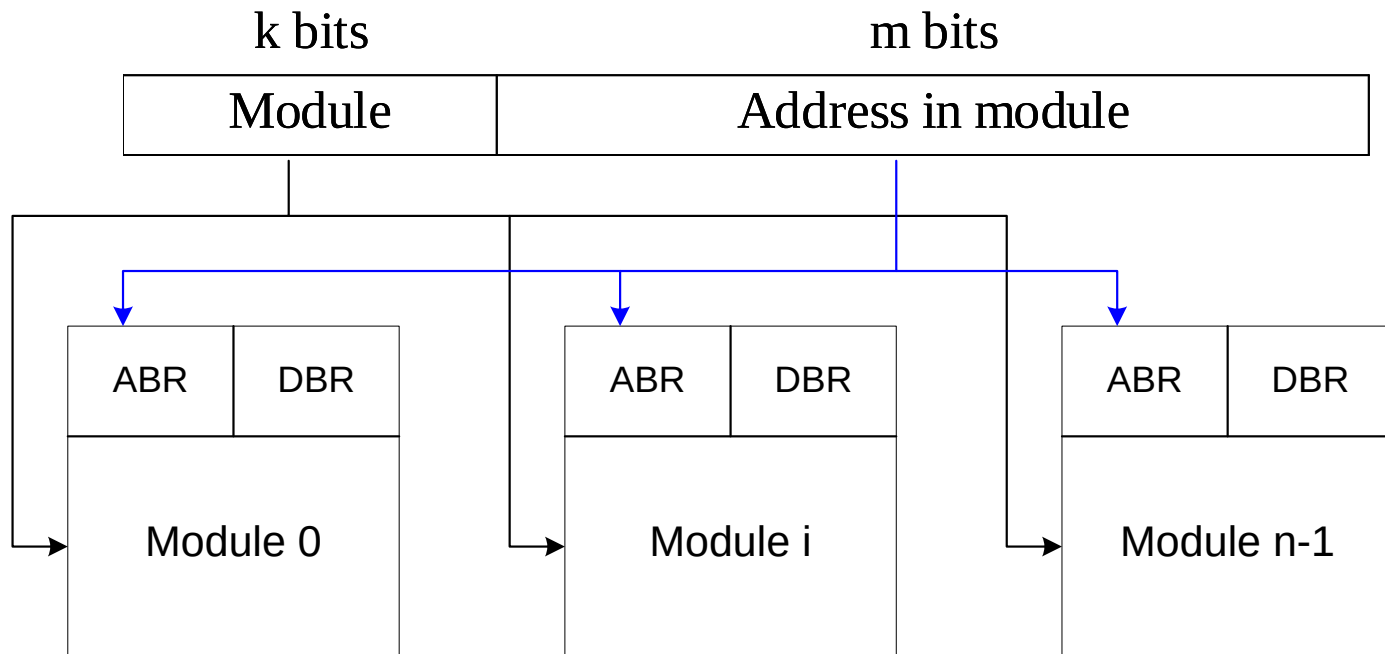
$$t_{avg} = 2.174 \text{ cycles}$$

Interleaving Concept

- The main memory is structured as a collection of physically separate modules, each with its own address buffer register (ABR) and data buffer register (DBR).
- Memory access operations may proceed in more than one module at the same time.
- Consecutive words in a module
 - If we want to transfer a block of data, only one module will be accessed, but other devices can DMA other modules.
- Consecutive words in consecutive module
 - In case of block access, processor can access several modules at the same time.

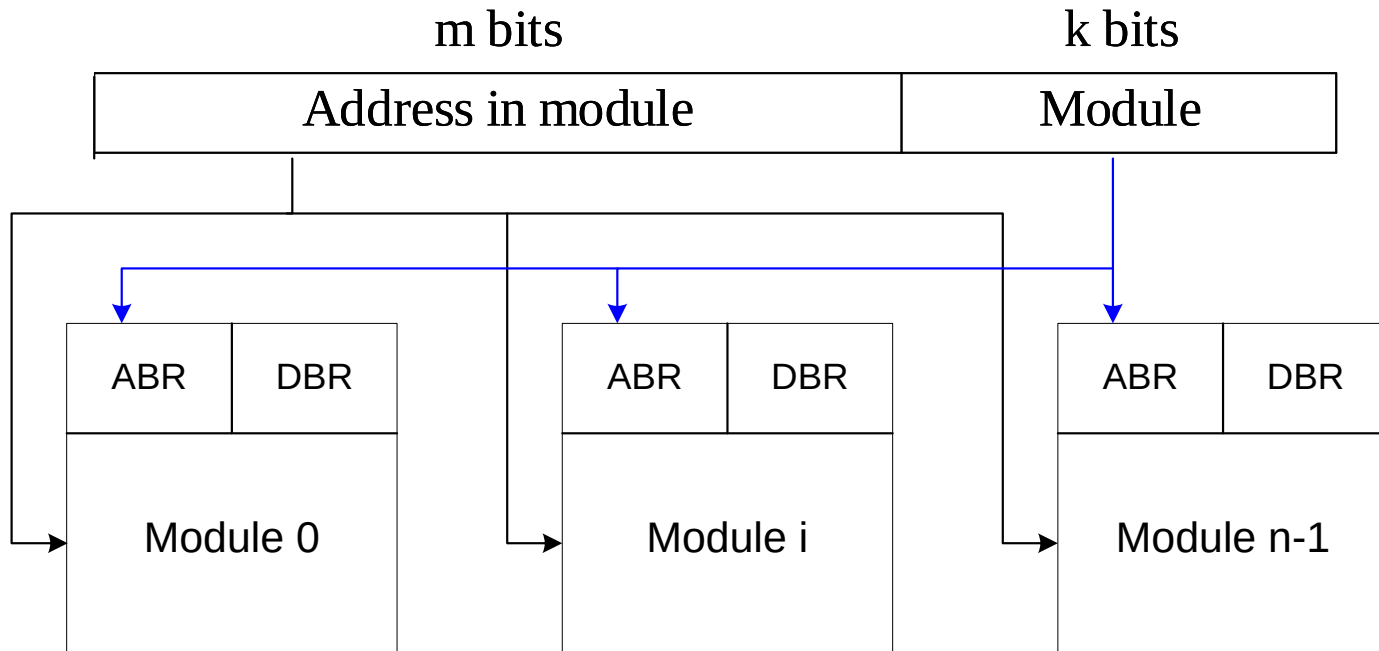
Memory Interleaving (1)

Consecutive words in a module



Memory Interleaving (2)

Consecutive words in consecutive modules



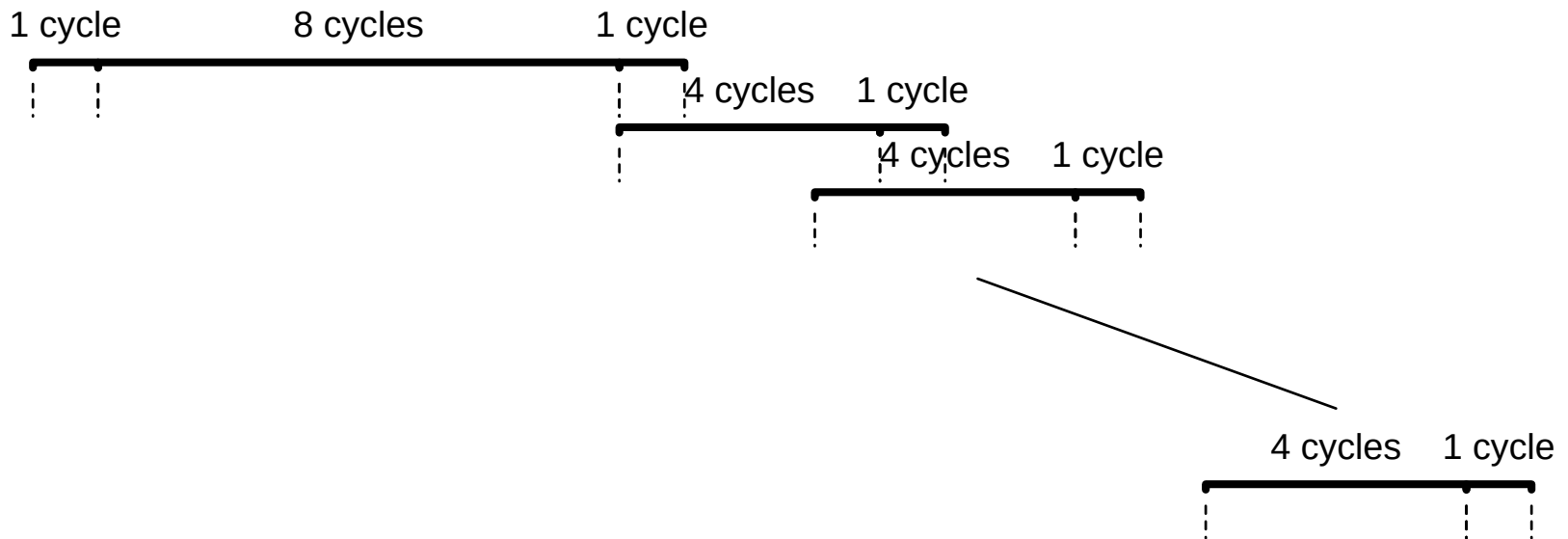
Example Problem for Memory Interleaving

- Assumes
- An 8-word block cache is used.
- Hardware takes 1 cycle to send an address to the memory controller.
- The time used to access the first word is 8 cycles.
- The time used to access one word is 4 cycles.
- The time used to send one word to a cache is 1 cycle.

Example Problem (1)

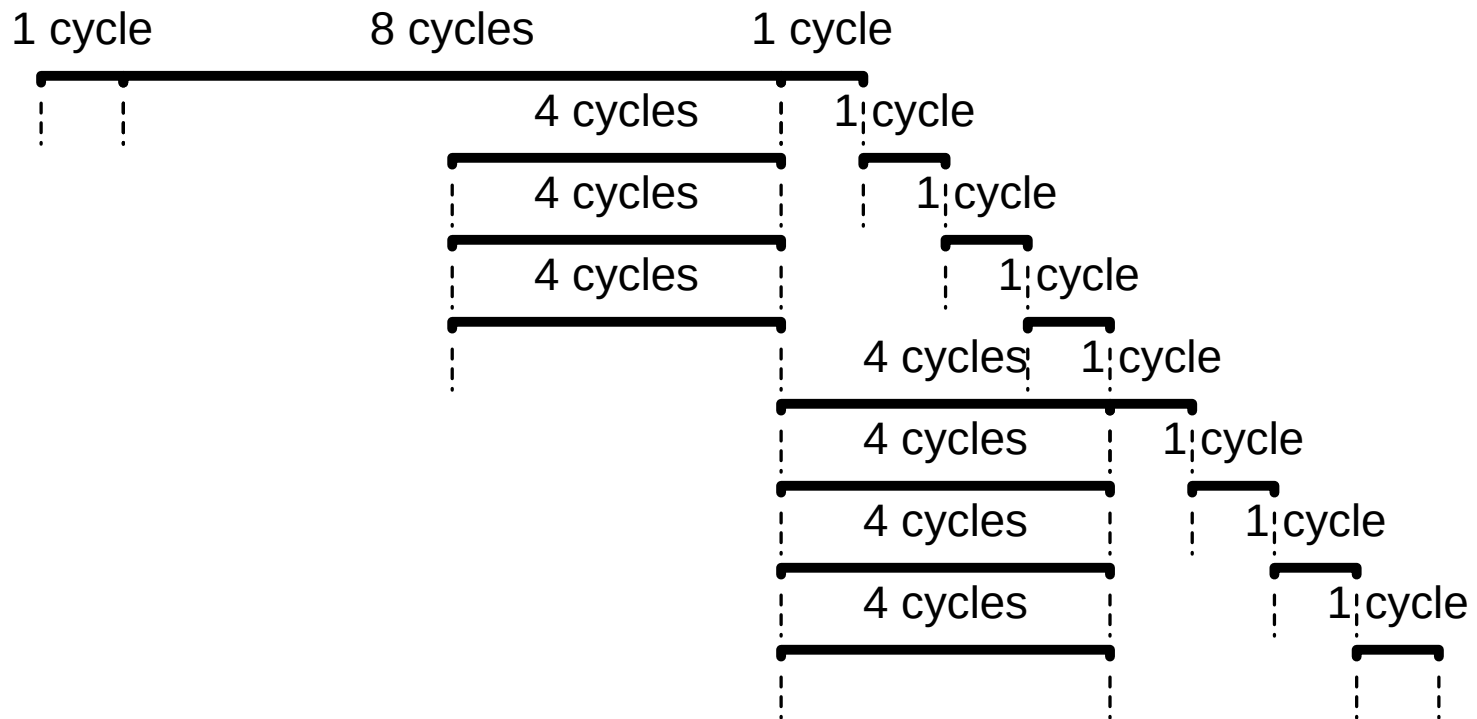
- An 8-word block cache is used.
- Hardware takes 1 cycle to send an address to the memory controller.
- The time used to access the first word is 8 cycles.
- The time used to access one word is 4 cycles.
- The time used to send one word to a cache is 1 cycle.

- Non interleaving memory
 - $T = 1 + 8 + (7 * 4) + 1 = 38$ cycles



Example Problem (2)

- 4-ways interleaving modules memory.
 - $T = 1 + 8 + 4 + 4 = 17$ cycles



Other Enhancements

- Write buffer

The CPU places each write request into a buffer and continues execution of the next instruction. (understand write though and write back)

- Pre-fetching

To avoid stalling of the CPU, it is possible to pre-fetch the data into the cache before it is needed. (software: special instruction and hardware: attempt to discover the pattern in reference memory)

- Lockup-free cache

A cache structure allows the CPU to access the cache while a miss is being serviced.

LAB

LRU Example

- Example

- Data cache has 8 blocks, each block contain a 16-bit word.
- A 16-bit memory address is used.
- Array $A(4, 10)$ is stored in column order.

Example Program

- Execute a program that normalizes the elements of the first row with respect to the average value of elements in the row.

```
SUM := 0
```

```
for j:=0 to 9 do
```

```
    SUM := SUM + A(0,j);
```

```
end
```

```
AVE := SUM / 10;
```

```
for i:=9 downto 0 do
```

```
    A(0,i) := A(0,i) / AVE;
```

```
end
```

Mapping Schemes

- Direct mapping
 - tag = 13, block = 3, word = 0
- Associative mapping
 - tag = 16, word = 0
- Set associative mapping
 - tag = 15, set = 1, word = 0

LRU with Direct mapping

- Direct mapping

Content of data cache after pass :									
Block position	j = 0								
0									
1									
2									
3									
4									
5									
6									
7									

Contents of a direct-mapped data cache.